

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ  
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ  
Ордена Трудового Красного Знамени федеральное государственное бюджетное  
образовательное учреждение высшего образования  
**«Московский технический университет связи и информатики»**

Кафедра «Программная инженерия»

# **ОТЧЁТ**

**по учебной практике**

**Тема: «Разработка интеллектуальной поисковой системы по  
внутренней базе знаний университета»**

Выполнил: студент группы БПИ2403  
Лихачёв Алексей

Проверила: Мосева М. С.

Москва, 2026

## 1 Цель и задачи практики

Цель учебной практики — закрепление и углубление теоретических знаний, полученных при изучении дисциплин направления «Программная инженерия», через разработку полнофункционального программного продукта с использованием современных технологий. В рамках практики командой из четырёх студентов разрабатывалась интеллектуальная поисковая система по внутренней базе знаний университета — веб-приложение «Кампус», объединяющее загрузку учебных материалов, полнотекстовый поиск, учебные очереди на сдачу работ и безопасную интеграцию с сервисами МТУСИ.

В соответствии с распределением ролей на организационном собрании автор отчёта выполнял роль бэкенд-инженера (BE). Индивидуальные задачи автора:

- спроектировать модель данных и настроить миграции базы данных (Alembic);
- реализовать аутентификацию и авторизацию (JWT, ролевая модель RBAC);
- реализовать загрузку, валидацию, парсинг и чанкинг документов PDF/DOCX;
- настроить полнотекстовый поиск в Elasticsearch с индексацией и историей запросов;
- реализовать серверную подсистему учебных очередей;
- подключить кеширование поиска в Redis и оформить документацию OpenAPI (Swagger).

## 2 Индивидуальное задание

Автор отвечал за бэкенд-часть системы: REST API, бизнес-логику, работу с базами данных, интеграцию с Elasticsearch и Redis. Ниже приведён перечень пунктов технического задания, закрытых автором.

Таблица 1 — Реализованные требования технического задания

| ID    | Требование                                                          | Роль |
|-------|---------------------------------------------------------------------|------|
| BE-01 | REST API эндпоинт POST /api/v1/documents/upload для загрузки файлов | BE   |
| BE-02 | Валидация файлов: формат (PDF, DOCX), размер до 20 МБ, HTTP 400     | BE   |

| ID    | Требование                                                               | Роль |
|-------|--------------------------------------------------------------------------|------|
| BE-03 | Генерация уникального идентификатора (UUID) документа                    | BE   |
| BE-04 | Извлечение текста: pdfplumber (PDF), python-docx (DOCX)                  | BE   |
| BE-05 | Разбиение текста на чанки по 1000 символов с перекрытием 100             | BE   |
| BE-06 | Настройка Elasticsearch с русским анализатором, индекс documents         | BE   |
| BE-07 | Индексация чанков с метаданными (file_name, page_number, chunk_id, text) | BE   |
| BE-08 | Эндпоинт GET /api/v1/search?q= с multi_match по полю text                | BE   |
| BE-09 | Результаты поиска в JSON (chunk_id, file_name, page, text, score)        | BE   |
| BE-10 | Кеширование частых запросов в Redis (TTL = 5 минут)                      | BE   |
| BE-A2 | Демо-вход и выпуск JWT-токена                                            | BE   |
| BE-A3 | Ролевая модель доступа (RBAC): student / teacher / admin                 | BE   |
| BE-Q  | Серверная подсистема учебных очередей (создание, запись, оценки)         | BE   |
| API   | Документирование эндпоинтов через OpenAPI 3.0 (Swagger UI на /docs)      | BE   |
| API   | Корректные HTTP-статусы 200/400/404/500                                  | BE   |

### 3 Ход выполнения работы

Работа велась по методологии ветвления Git: каждая задача разрабатывалась в отдельной ветке и вливалась в ветку разработки через Pull Request (PR) после ревью. Всего автором подготовлено и слито 6 запросов на слияние; суммарный вклад составил свыше 8000 добавленных строк кода. Ниже описаны основные группы выполненных работ.

#### 3.1 Модель данных и миграции

Спроектирована модель пользователя (User) и настроена инфраструктура миграций базы данных на Alembic. Созданы миграции для таблиц пользователей, документов, истории поиска и очередей, а также сид демонстрационных пользователей. Для доступа к PostgreSQL используется SQLAlchemy в асинхронном режиме.

#### 3.2 Аутентификация и авторизация

Реализован демонстрационный вход с выпуском JWT-токена и ролевая модель доступа (RBAC) с тремя ролями: студент, преподаватель, администратор. Проверка прав вынесена в зависимости FastAPI; защищённые эндпоинты принимают заголовок Authorization: Bearer. Логика безопасности покрыта юнит-тестами.

### 3.3 Загрузка, парсинг и чанкинг документов

Реализован эндпоинт загрузки документов с валидацией формата (PDF, DOCX) и размера (до 20 МБ) и генерацией уникального идентификатора (UUID). Извлечение текста выполняется постранично: pdfplumber для PDF и python-docx для DOCX. Извлечённый текст разбивается на чанки по 1000 символов с перекрытием 100 символов между соседними сегментами, что соответствует требованиям BE-04 и BE-05.

### 3.4 Полнотекстовый поиск в Elasticsearch

Настроен индекс documents в Elasticsearch с русскоязычным анализатором. Каждый чанк документа индексируется с метаданными (file\_name, page\_number, chunk\_id, text). Реализован эндпоинт GET /api/v1/search с запросом multi\_match по полю text; результаты возвращаются в формате JSON с оценкой релевантности (score) и ранжированием. Дополнительно сохраняется история поисковых запросов.

### 3.5 Подсистема учебных очередей

Реализована серверная логика учебных очередей: создание очередей преподавателем и администратором, запись и выход студентов, выставление оценок, а также сид очереди практической защиты. Обеспечены отложенная загрузка участников очереди и уникальность пары (queue\_id, student\_id).

### 3.6 Кеширование и документация API

Подключено кеширование результатов поиска в Redis с TTL 5 минут; ключ кеша учитывает идентификатор и роль пользователя. Все операции с кешем отказоустойчивы: при недоступности Redis приложение продолжает работать без кеша. Эндпоинты задокументированы по стандарту OpenAPI 3.0, интерактивная документация Swagger UI доступна по адресу /docs.

Таблица 2 — Запросы на слияние (Pull Requests) автора

| № PR | Описание                                          | Дата       |
|------|---------------------------------------------------|------------|
| 62   | Модель User и начальная миграция Alembic          | 01.07.2026 |
| 63   | Демо-вход, JWT и RBAC [BE-A2, BE-A3]              | 01.07.2026 |
| 64   | Документы — загрузка, парсинг, чанкинг [BE-D1–D4] | 01.07.2026 |

| № PR | Описание                                                     | Дата       |
|------|--------------------------------------------------------------|------------|
| 65   | Elasticsearch-поиск, индексация, история запросов [BE-S1–S4] | 01.07.2026 |
| 71   | Система учебных очередей [BE-Q1–Q4, BE-P1]                   | 04.07.2026 |
| 72   | Swagger-документация и Redis-кэш [DOC-02, BE-S5]             | 04.07.2026 |

## 4 Исходный код программы

Ниже приведены характерные фрагменты разработанного кода. Листинги оформлены шрифтом Consolas, 10 пунктов.

Листинг 1 — Парсинг и чанкинг документов (backend/app/services/document\_parser.py)

```
def extract_text(content: bytes, file_type: str) -> list[tuple[int, str]]:
    """Возвращает список (номер_страницы, текст). PDF или DOCX."""
    try:
        if file_type.upper() == "PDF":
            return _extract_pdf(content)
        return _extract_docx(content)
    except ParserError:
        raise
    except Exception as exc:
        raise ParserError(str(exc)) from exc
def _extract_pdf(content: bytes) -> list[tuple[int, str]]:
    pages: list[tuple[int, str]] = []
    with pdfplumber.open(io.BytesIO(content)) as pdf:
        if not pdf.pages:
            raise ParserError("PDF не содержит страниц")
        for i, page in enumerate(pdf.pages, start=1):
            pages.append((i, page.extract_text() or ""))
    return pages
```

Листинг 2 — Отказоустойчивый кэш поиска в Redis (backend/app/services/cache.py)

```
async def cache_get(key: str) -> Any | None:
    """Вернуть десериализованное значение из кэша или None."""
    try:
        raw = await _get_client().get(key)
        return json.loads(raw) if raw is not None else None
    except Exception:
        log.debug("Redis cache_get miss/error: %s", key)
        return None
async def cache_set(key: str, value: Any, ttl: int = 300) -> None:
    """Сохранить JSON-сериализуемое значение в кэш с TTL."""
    try:
        await _get_client().set(key, json.dumps(value), ex=ttl)
    except Exception:
        log.debug("Redis cache_set error: %s", key)
```

## 5 Заключение

В ходе учебной практики автором в составе команды разработана интеллектуальная поисковая система по внутренней базе знаний университета. Лично автором выполнены следующие задачи:

- спроектирована модель данных и настроены миграции Alembic для PostgreSQL;
- реализованы аутентификация по JWT и ролевая модель доступа (RBAC);
- реализованы загрузка, валидация, парсинг и чанкинг документов PDF/DOCX;
- настроен полнотекстовый поиск в Elasticsearch с индексацией, ранжированием и историей запросов;
- реализована серверная подсистема учебных очередей;
- подключено кеширование поиска в Redis и оформлена документация OpenAPI (Swagger).

Поставленные индивидуальные задачи выполнены в полном объеме. В процессе работы закрепились навыки проектирования REST API на FastAPI, работы с реляционными базами данных и поисковым движком Elasticsearch, а также командной разработки с использованием системы контроля версий Git и код-ревью через Pull Request. Итоговый программный продукт собирается и запускается одной командой через Docker Compose и соответствует требованиям технического задания.